CSE 4631

Digital Signal Processing

Week 15

Moving Average Filters

- The most common filter in DSP
- **It is optimal for a common task:** reducing noise while retaining sharp step response.
- A great filter for time domain encoded signals, the worst filter for frequency domain encoded signals
    - Almost unable to separate one band of frequencies from another
- Relatives of the moving average filter include the **Gaussian**, **Blackman**, and **multiple-pass** moving average which have slightly better performance in the frequency domain, at the expense of increased computation time.

Implementation by Convolution

- Operates by averaging a number of points from the input signal to produce each point in the output signal
- In equation form, this is written:

$$y[i] = \frac{1}{M} \sum_{j=0}^{M-1} x[i+j]$$

  - $x[\ ]$ is the input signal
  - $y[\ ]$ is the output signal
  - M is the number of points in the average = risetime

Implementation by Convolution (contd.)

- As an alternative, the group of points from the input signal can be chosen *symmetrically* around the output point – Symmetrical averaging
- Symmetrical averaging requires that M be an odd number

$$y[i] = \frac{1}{M} \sum_{j=\frac{-(M-1)}{2}}^{\frac{(M-1)}{2}} x[i+j]$$

- So, for a 5 point moving average filter, j will run from 0 to 4 (one side averaging) or -2 to 2 (symmetrical averaging).
- Programming is slightly easier with the points on only one side – one side averaging
- One side averaging produces a relative shift between the input and output signals

Implementation by Convolution (contd.)

$$y[2] = \frac{1}{3}(x[2] + x[3] + x[4])$$

x(n)		$x[0]$	$x[1]$	$x[2]$	$x[3]$	$x[4]$	$x[5]$
$y[0] = (x[0] + x[1] + x[2])/3$	0	1/3	1/3	1/3	0	0	0
$y[1] = (x[1] + x[2] + x[3])/3$	0	0	1/3	1/3	1/3	0	0
$y[2] = (x[2] + x[3] + x[4])/3$	0	0	0	1/3	1/3	1/3	0
$y[3] = (x[3] + x[4] + x[5])/3$	0	0	0	0	1/3	1/3	1/3

Table: Convolution table for **moving average filter (one side averaging)**

Implementation by Convolution (contd.)

$$y[2] = \frac{1}{3}(x[1] + x[2] + x[3])$$

x(n)		$x[0]$	$x[1]$	$x[2]$	$x[3]$	$x[4]$	$x[5]$
$y[1] = (x[0] + x[1] + x[2])/3$	0	1/3	1/3	1/3	0	0	0
$y[2] = (x[1] + x[2] + x[3])/3$	0	0	1/3	1/3	1/3	0	0
$y[3] = (x[2] + x[3] + x[4])/3$	0	0	0	1/3	1/3	1/3	0
$y[4] = (x[3] + x[4] + x[5])/3$	0	0	0	0	1/3	1/3	1/3

Table: Convolution table for **moving average filter (symmetrical averaging)**

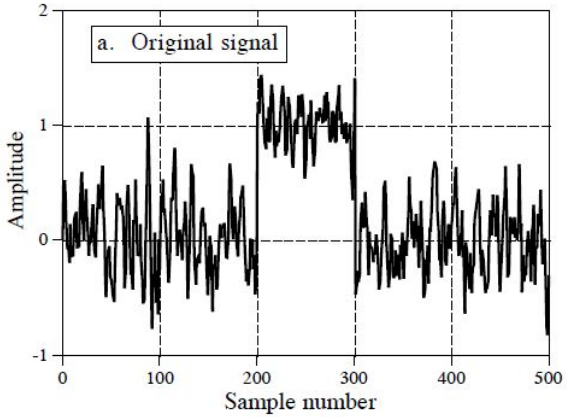
Implementation by Convolution (contd.)

- So, a 5 point filter has the filter kernel: 0, 0, 1/5, 1/5, 1/5, 1/5, 1/5, 0, 0.
- That is, **the moving average filter is a convolution of the input signal with a rectangular pulse having an area of one.**

```
100 'MOVING AVERAGE FILTER
110 'This program filters 5000 samples with a 101 point moving
120 'average filter, resulting in 4900 samples of filtered data.
130 '
140 DIM X[4999]                    'X[ ] holds the input signal
150 DIM Y[4999]                    'Y[ ] holds the output signal
160 '
170 GOSUB XXXX                     'Mythical subroutine to load X[ ]
180 '
190 FOR I% = 50 TO 4949            'Loop for each point in the output signal
200   Y[I%] = 0                    'Zero, so it can be used as an accumulator
210   FOR J% = -50 TO 50           'Calculate the summation
220     Y[I%] = Y[I%] + X(I%+J%]
230   NEXT J%
240   Y[I%] = Y[I%]/101            'Complete the average by dividing
250 NEXT I%
260 '
270 END
```

TABLE 15-1

Noise Reduction vs Step Response



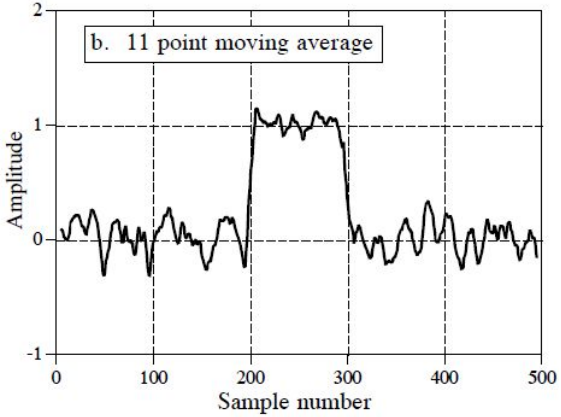
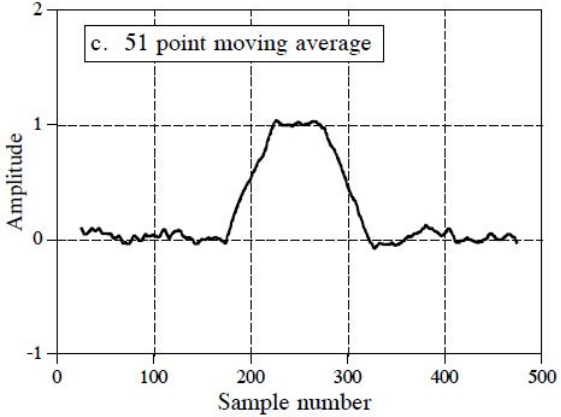


FIGURE 15-1
Example of a moving average filter. In (a), a rectangular pulse is buried in random noise. In (b) and (c), this signal is filtered with 11 and 51 point moving average filters, respectively. As the number of points in the filter increases, the noise becomes lower; however, the edges becoming less sharp. The moving average filter is the *optimal* solution for this problem, providing the lowest noise possible for a given edge sharpness.

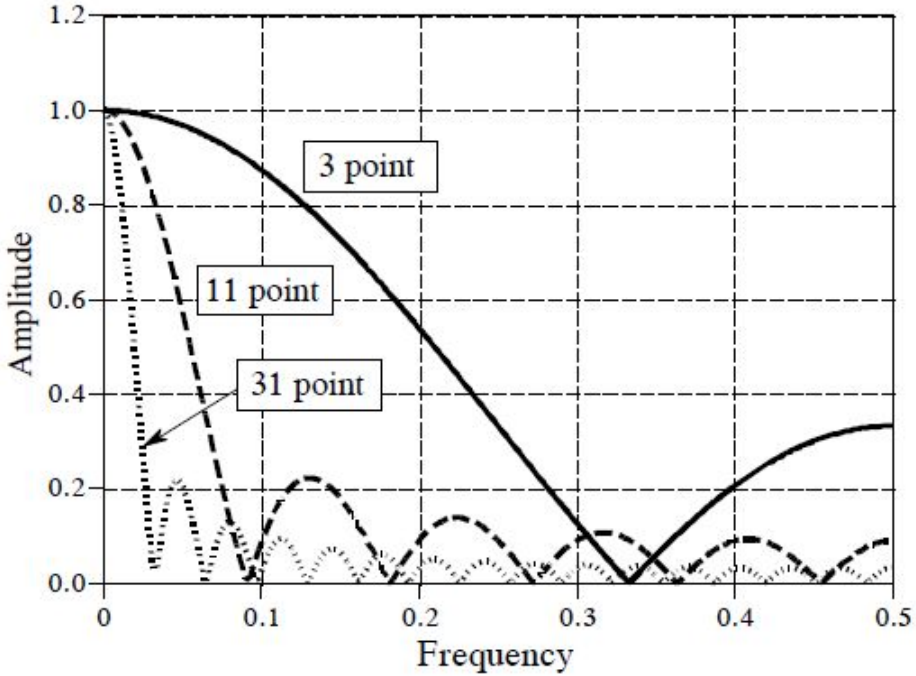


Noise Reduction vs Step Response (contd.)

- The smoothing action of the moving average filter decreases the amplitude of the random noise (good), but also reduces the sharpness of the edges (bad).
- Small $M \rightarrow$ fast rise time, less smoothing (more noise), but able to capture sharp timing / events happening in small intervals
- Large $M \rightarrow$ slow rise time, more smoothing (less noise), but blurs/merges events that happen close together.
- Of all the possible linear filters that could be used, the moving average produces the lowest noise for a given edge sharpness.
- The amount of noise reduction is equal to the square-root of the number of points in the average.
- For example, a 100 point moving average filter reduces the noise by a factor of 10.

Moving Average Filters: Frequency Response

FIGURE 15-2
Frequency response of the moving average filter. The moving average is a very poor low-pass filter, due to its slow roll-off and poor stopband attenuation. These curves are generated by Eq. 15-2.



Moving Average Filters: Frequency Response (contd.)

- The roll-off is very slow
- The stopband attenuation is ghastly
- This filter cannot separate one band of frequencies from another
- Therefore, moving average is an exceptionally good smoothing filter (the action in the time domain), but an exceptionally bad low-pass filter (the action in the frequency domain).

EQUATION 15-2
Frequency response of an M point moving average filter. The frequency, f, runs between 0 and 0.5. For $f = 0$, use: $H[f] = 1$

$$H[f] = \frac{\sin(\pi f M)}{M \sin(\pi f)}$$

Relatives of the Moving Average Filter

- In a perfect world, we would only have to deal with time domain or frequency domain encoded information, but never a mixture of the two in the same signal
- Unfortunately, there are some applications where both domains are simultaneously important
- Relatives of the moving average filter have better frequency domain performance, and can be useful in these mixed domain applications.

Multiple-pass moving average filters

- It involves passing the input signal through a moving average filter two or more times
- Two passes are equivalent to using a triangular filter kernel (a rectangular filter kernel convolved with itself)
- After four or more passes, the filter kernel looks like a Gaussian

Relatives of the Moving Average Filter (contd.)

Multiple-pass moving average filters (contd.)

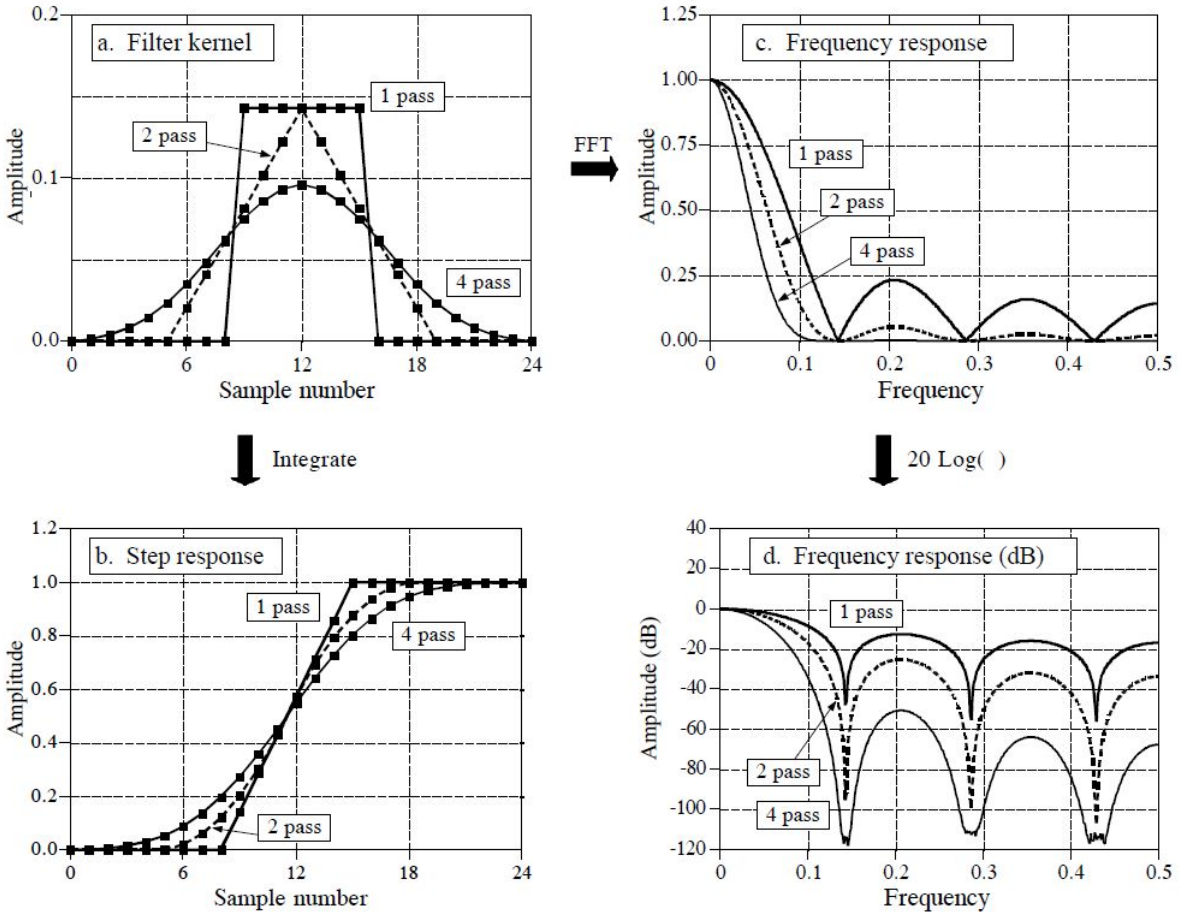


FIGURE 15-3
Characteristics of multiple-pass moving average filters. Figure (a) shows the filter kernels resulting from passing a seven point moving average filter over the data once, twice and four times. Figure (b) shows the corresponding step responses, while (c) and (d) show the corresponding frequency responses.

Relatives of the Moving Average Filter (contd.)

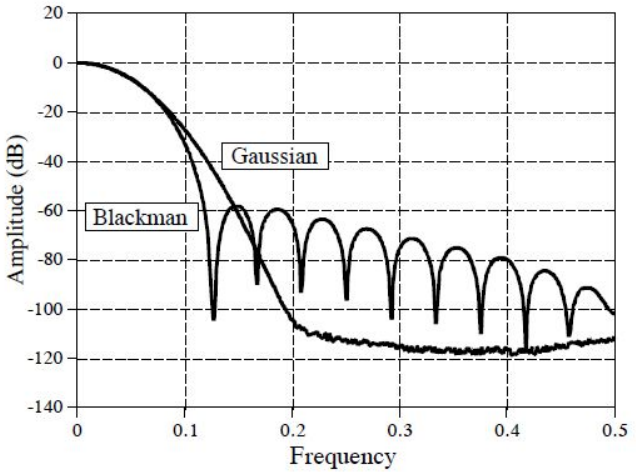
Gaussian filters

- A pure gaussian is used as a filter kernel, the frequency response if also a Gaussian

Blackman filters

- Uses blackman window as a filter kernel

FIGURE 15-4
Frequency response of the Blackman window and Gaussian filter kernels. Both these filters provide better stopband attenuation than the moving average filter. This has no advantage in removing random noise from time domain encoded signals, but it can be useful in mixed domain problems. The disadvantage of these filters is that they must use *convolution*, a terribly slow algorithm.



Relatives of the Moving Average Filter (contd.)

How are these relatives of the moving average filter better than the moving average filter itself?

Three ways:

1. They have *better stopband attenuation* than the moving average filter
2. The *filter kernel tapers to a smaller amplitude near the ends*
    - Recall that each point in the output signal is a weighted sum of a group of samples from the input.
    - If the filter kernel tapers, samples in the input signal that are farther away are given less weight than those close by.
3. The *step responses are smooth curves*, rather than the abrupt straight line of the moving average.

Relatives of the Moving Average Filter (contd.)

Differences between these filters and moving average filter

- The moving average filter and its relatives are all about the same at reducing random noise while maintaining a sharp step response.
- The ambiguity lies in how the risetime of the step response is measured.
  - If the risetime is measured from 0% to 100% of the step, the moving average filter is the best you can do, as previously shown.
  - Measuring the risetime from 10% to 90% makes the Blackman window better than the moving average filter.
- However, this is just theoretical, you can consider these filters equal in this parameter
- **The biggest difference is execution speed**.
  - Moving average filter runs very fast using a recursive algorithm which makes it the fastest digital filter available.
  - Multiple passes of the moving average filter will be correspondingly slower, but still very quick
  - The Gaussian and Blackman filters are excruciatingly slow, because they *must* use convolution

Recursive Implementation

- Imagine passing an input signal, $x[\ ]$, through a seven point moving average filter to form an output signal, $y[\ ]$
- Two adjacent points $y[50]$ and $y[51]$ are calculated as:

$$y[50] = x[47] + x[48] + x[49] + x[50] + x[51] + x[52] + x[53]$$

$$y[51] = x[48] + x[49] + x[50] + x[51] + x[52] + x[53] + x[54]$$

- Instead of recalculating the highlighted part for $y[51]$, we can calculate $y[51]$ more efficiently as: $y[51] = y[50] + x[54] - x[47]$
- Once $y[51]$ has been found using $y[50]$, then $y[52]$ can be calculated from $y[51]$, and so on.

Recursive Implementation (contd.)

- This can be expressed in the equation:

$$y[i] = y[i - 1] + x[i + p] - y[i - q]$$

Where, $p = (M - 1)/2$

$q = p + 1$

- This algorithm is faster for the following four reasons:
  - Only two computations per point, regardless of the length of the filter kernel.
  - Addition and subtraction are the only math operations needed, while most digital filters require time-consuming multiplication.
  - The indexing scheme is very simple. Each index is found by adding or subtracting integer constants (i.e., p and q) that can be calculated before the filtering starts.
  - The entire algorithm can be carried out with integer representation. Depending on the hardware used, integers can be more than an order of magnitude faster than floating point.

Recursive Implementation (contd.)

```
100 'MOVING AVERAGE FILTER IMPLEMENTED BY RECURSION
110 'This program filters 5000 samples with a 101 point moving
120 'average filter, resulting in 4900 samples of filtered data.
130 'A double precision accumulator is used to prevent round-off drift.
140 '
150 DIM X[4999]                    'X[ ] holds the input signal
160 DIM Y[4999]                    'Y[ ] holds the output signal
170 DEFDBL  ACC                    'Define the variable ACC to be double precision
180 '
190 GOSUB XXXX                     'Mythical subroutine to load X[ ]
200 '
210 ACC = 0                        'Find Y[50] by averaging points X[0] to X[100]
220 FOR I% = 0 TO 100
230   ACC = ACC + X[I%]
240 NEXT I%
250 Y[50] = ACC/101
260 '                              'Recursive moving average filter (Eq. 15-3)
270 FOR I% = 51 TO 4949
280   ACC = ACC + X[I%+50] - X[I%-51]
290   Y[I%] = ACC/101
300 NEXT I%
310 '
320 END
```

TABLE 15-2

Recursive Implementation (contd.)

- The moving average recursive filter is very different from typical recursive filters.
  - Most recursive filters have an infinitely long impulse response (IIR), composed of sinusoids and exponentials
  - Impulse response of the moving average is a rectangular pulse (finite impulse response, or FIR)